STRUTTURA DATI: ALBERO HEAP

UN ALBERO HEAP È UN ALBERO BINARIO COMPLETO (NB: COMPLETO, NON PIENO) CARATTERIZZATO DA 3 PROPRIETÀ FONDAMENTALI.

1. FORMA: LE FOGLIE SI TROVANO TUTTE A PROFONDITÀ h O $h-1$
2. GRADI: TUTTI I NODI INTERNI, TRANNE AL PIÙ UNO, HANNO GRADO 2
3. ORDINAMENTO PARZIALE (MAX-HEAP): PER OGNI NODO i, IL VALORE DEL NODO PADRE NON È MAI INFERIORE AL VALORE DEI FIGLI. IL VALORE MASSIMO SI TROVA SEMPRE NELLA RADICE

RAPPRESENTAZIONE SU ARRAY

UN ALBERO BINARIO COMPLETO SI MAPPA NATURALMENTE SU UN ARRAY A, SCANSIONANDO I NODI LIVELLO PER LIVELLO DA SX VERSO DX.

- LA RADICE SI TROVA IN POSIZIONE A[1] (O A[0] IN RAPPRESENTAZIONI 0-BASED)
- SE UN NODO SI TROVA ALL'INDICE i, I SUOI "PARENTI" SI CALCOLANO IN TEMPO $\Theta(1)$

  PARENT(i) = $\lfloor i/2 \rfloor$ (GENITORE)

  LEFT(i) = $[2i]$ (FIGLIO SX)

  RIGHT(i) = $[2i+1]$ (FIGLIO DX)

- NODI INTERNI E FOGLIE: I NODI INTERNI A UN HEAP DI n ELEMENTI SI TROVANO AGLI INDICI $1 \leq i \leq \lfloor n/2 \rfloor$, MENTRE LE FOGLIE SI TROVANO DA $\lfloor n/2 \rfloor + 1$ FINO A n

MANTENIMENTO DELLA PROPRIETÀ DI HEAP: LA PROCEDURA HEAPIFY

HEAPIFY È LA PROCEDURA RICORSIVA CHE AGGIUSTA L'ALBERO. SE UN NODO i VIOLA LA PROPRIETÀ MAX-HEAP (È PIÙ PICCOLO DEI SUOI DISCENDENTI), HEAPIFY LO FA SCENDERE SCAMBIANDOLO CON IL FIGLIO MAGGIORE, RIPETENDO IL PROCESSO FINCHÉ L'ALBERO NON TORNA AD ESSERE UN HEAP.

NB: HEAPIFY(A, i) FUNZIONA SOLO SE I SOTTOALBERI RADICATI IN LEFT(i) E RIGHT(i) SONO GIÀ DEI MAX-HEAP VALIDI

NB: HEAPSIZE IDENTIFICA IL NUMERO DI NODI DELL'ALBERO HEAP CORRETTAMENTE MEMORIZZATI NEL VETTORE, QUINDI

$$0 \leq \text{HEAPSIZE} \leq A.\text{LENGTH}$$

```
HEAPIFY(A, i)
    LEFT = 2 · i
    RIGHT = 2 · i + 1          } INDICI
    LARGEST = i
    IF LEFT ≤ HEAPSIZE && A[LEFT] > A[i]
        LARGEST = LEFT
    IF RIGHT ≤ HEAPSIZE && A[RIGHT] > A[LARGEST]
        LARGEST = RIGHT
    IF LARGEST != i
        SWAP(A, i, LARGEST)
        HEAPIFY(A, LARGEST)
```

COSTO: LA DISCESA COMPIE AL PIÙ $h = \log_2 n$ PASSI. IL CASO PEGGIORE È $O(\lg n)$

LA COSTRUZIONE BOTTOM-UP: BUILD-HEAP

PER TRASFORMARE UN ARRAY DISORDINATO IN UN MAX HEAP SI UTILIZZA HEAPIFY PARTENDO DAL BASSO VERSO L'ALTO, POICHÉ LE FOGLIE SONO GIÀ DEGLI HEAP VALIDI DI DIMENSIONE 1. SI PUÒ QUINDI IGNORARE TUTTA LA SECONDA METÀ DELL'ARRAY, INIZIANDO A CHIAMARE HEAPIFY A RITROSO, PARTENDO DALL'ULTIMO NODO INTERNO $i = \lfloor i/2 \rfloor$, FINO ALLA RADICE $i = 1$ (O $i = 0$)

```
BUILD-HEAP(A, n)
    HEAPSIZE = n
    FOR i = FLOOR(n/2) DOWN TO 1
        HEAPIFY(A, i)
```

COSTO: IL TEMPO DI HEAPIFY DIPENDE DALL'ALTEZZA DEL NODO, LA MAGGIOR PARTE DEI NODI SI TROVA AI LIVELLI BASSI DELL'ALBERO (ALTEZZA PICCOLA → HEAPIFY VELOCE). SOMMANDO I COSTI SU TUTTI I LIVELLI E SFRUTTANDO LA SERIE GEOMETRICA CONVERGENTE $\sum_{h=0}^{\infty} \frac{h}{2^h} = 2$, IL COSTO TOTALE COLLASSA A UNA COSTANTE MOLTIPLICATA PER n, OVVERO $O(n)$

L'ALGORITMO HEAP SORT

UNA VOLTA COSTRUITO IL MAX HEAP, L'ORDINAMENTO DIVENTA UN BANALE PROCESSO ITERATIVO DI ESTRAZIONE DEL MASSIMO:

1. SI PRELEVA IL MASSIMO (SITUATO SEMPRE IN RADICE) E LO SI POSIZIONA IN FONDO ALL'ARRAY, SCAMBIANDOLO CON A[HEAPSIZE]
2. SI "CONGELA" L'ELEMENTO ESTRATTO RIDUCENDO LA DIMENSIONE LOGICA (HEAPSIZE = HEAPSIZE - 1)
3. SI INVOCA HEAPIFY(A, 1) PER FAR SPROFONDARE LA RADICE, RIPRISTINANDO LA PROPRIETÀ MAX-HEAP

HEAPSORT(A, n)

```
BUILD-HEAP(A, n)
FOR CURR_END = n DOWN TO 2 DO
    SWAP(A, 1, CURR_END)
    HEAPSIZE = HEAPSIZE - 1
    HEAPIFY(A, 1)
```

COSTO: LA COSTRUZIONE INIZIALE COSTA $O(n)$. IL CICLO FOR ESEGUE $n-1$ CHIAMATE A HEAPIFY (CIASCUNA $O(\log n)$) IL TEMPO COMPLESSIVO PER CASO MIGLIORE, MEDIO, PEGGIORE È $O(n \log n)$